UrbanFlow CDMX

VialAI

Sistema de Predicción Estocástica de Tiempos de Traslado en la Zona Metropolitana del Valle de México

mediante Cadenas de Markov, Simulación Monte Carlo
e Inteligencia Artificial Conversacional

Carlos Armando López Encino

cale.delta@gmail.com

Diplomado en Ciencia de Datos | Generación 30
FES Acatlán | UNAM

Proyecto Integrador

Abril 2026

Resumen

UrbanFlow CDMX es un sistema de predicción probabilística de tiempos de viaje para la Zona Metropolitana del Valle de México (ZMVM) que combina *cadenas de Markov de tiempo discreto* con *simulación Monte Carlo* de $N = 10{,}000$ trayectorias por consulta. El sistema ingiere datos en tiempo real de cinco fuentes (TomTom Traffic Stats, TomTom Routing, OpenWeatherMap, C5 CDMX y Google Maps), construye un modelo estocástico del comportamiento del tráfico, aplica perturbaciones contextuales (cierres de infraestructura, eventos masivos, clima adverso) sobre la matriz de transición, y entrega al usuario las bandas de incertidumbre P_{10}, P_{50} y P_{90} del tiempo de viaje junto con un **Índice de Confiabilidad (IC)** que resume la volatilidad de la ruta. Sobre este motor se construyó **VialAI**, un agente conversacional basado en la API de Anthropic que traduce los resultados numéricos a explicaciones accionables para el usuario final.

A diferencia de los sistemas de navegación convencionales que producen una estimación puntual del tiempo de viaje, UrbanFlow CDMX cuantifica explícitamente la incertidumbre inherente al tráfico urbano, habilitando decisiones racionales de planeación logística. El nicho objetivo es la **logística B2B de última milla en la ZMVM** —un mercado que se proyecta alcance USD 15,630 millones en 2025 según Mordor Intelligence— donde el costo de un retraso por hora es tangible y la predictibilidad es más valiosa que la velocidad promedio. El sistema fue validado contra la API de TomTom Routing como ground truth, reportando un MAPE del P_{50} del 11.9 % sobre 30 rutas representativas de la ZMVM (7.1 puntos porcentuales mejor que el ETA de TomTom) y una cobertura empírica de la banda P_{10}–P_{90} del 93.3 %, superior a la cobertura nominal del 80 %.

Palabras clave: cadenas de Markov, simulación Monte Carlo, tráfico urbano, ZMVM, predicción probabilística, agentes LLM, logística de última milla, Index de Confiabilidad.

Índice

1 Motivación y Contexto **5**
1.1 El problema del tráfico en la ZMVM . . . 5
1.2 La limitación del enfoque determinístico . . . 6
1.3 Propuesta de valor . . . 6

2 Arquitectura General del Sistema **6**
2.1 Diagrama de componentes . . . 6
2.2 Stack tecnológico . . . 7

3 Fuentes de Datos e Ingesta **8**
3.1 TomTom Traffic Stats API . . . 8
3.2 TomTom Routing API . . . 9
3.3 OpenWeatherMap API . . . 9

4 Marco Teórico: Cadenas de Markov **9**
4.1 Definición y propiedad de Markov . . . 9
4.2 Espacio de estados en UrbanFlow CDMX . . . 10
4.3 Matriz de transición . . . 10
4.4 Distribución en k pasos . . . 10
4.5 Distribución estacionaria . . . 11
4.6 Estimación de la matriz desde datos históricos . . . 11

5 Implementación de la Cadena de Markov **11**
5.1 Clase `MarkovTrafficChain` . . . 11
5.2 Matriz calibrada para hora pico en la ZMVM . . . 12

6 Marco Teórico: Simulación Monte Carlo **12**
6.1 Fundamento matemático . . . 12
6.2 Distribución de velocidades por estado . . . 13
6.3 Modelo de tiempo de viaje . . . 13

7 Implementación del Motor Monte Carlo **14**
7.1 Clase `MonteCarloEngine` . . . 14
7.2 Interpolación lineal sub-paso . . . 14
7.3 Estructura del resultado . . . 14

8 Pipeline Integrador en Tiempo Real **15**
8.1 Inferencia del estado inicial de tráfico . . . 15
8.2 Ajuste climático de los parámetros de velocidad . . . 15
8.3 Flujo completo de predicción . . . 15
8.4 Perturbaciones contextuales . . . 16
8.5 Extensión: detección de eventos en cuasi-tiempo-real . . . 17

9 Agente VialAI: Interfaz Conversacional **18**
9.1 Arquitectura del agente . . . 18
9.2 Structured Outputs con Pydantic . . . 19
9.3 Function Calling: las tres herramientas de VialAI . . . 19
9.4 Integración en Streamlit . . . 19

10 Resultados y Validación **20**
10.1 Convergencia del estimador Monte Carlo . . . 20
10.2 Bandas de incertidumbre e Índice de Confiabilidad por estado . . . 20
10.3 Validación empírica: MAPE y cobertura de banda . . . 21
10.4 Casos testigo: ganado y perdido . . . 22

11 Estrategia de Negocio: VialAI para Logística B2B de Última Milla **22**
11.1 Diagnóstico sectorial . . . 22
11.2 Usuarios objetivo y sus necesidades . . . 24
11.3 Accionables habilitados por VialAI . . . 24
11.4 Propuesta de valor y modelo de entrega . . . 25

12 Conclusiones y Trabajo Futuro **25**
12.1 Conclusiones . . . 25
12.2 Retos encontrados durante el desarrollo . . . 26
12.3 Trabajo futuro . . . 26

A Parámetros de configuración del motor **28**

B Glosario **28**

1 Motivación y Contexto

1.1 El problema del tráfico en la ZMVM

La Zona Metropolitana del Valle de México concentra a más de **21.8 millones de habitantes** [1] distribuidos en las 16 alcaldías de la Ciudad de México y 59 municipios conurbados del Estado de México, además de uno del estado de Hidalgo [2]. Esta aglomeración la coloca sistemáticamente entre las zonas urbanas más congestionadas del mundo.

De acuerdo con el *TomTom Traffic Index 2024*, la CDMX ocupa la primera posición mundial en nivel de congestión con un índice del 52 %, lo que significa que los viajes en automóvil duran en promedio 52 % más de lo que durarían con vías libres [3]. En términos operativos:

- Los conductores pierden en promedio **152 horas al año** atrapados en tráfico, equivalentes a más de seis días completos de 24 horas o 19 jornadas laborales de 8 horas [4].
- Recorrer 10 kilómetros toma en promedio **31 minutos 53 segundos**, a una velocidad media de 18.8 km/h. En condiciones ideales ese mismo trayecto se realizaría en 8 minutos [5].
- En horas pico, la velocidad en las principales arterias (Eje Central, Viaducto, Periférico, Circuito Interior) puede caer hasta **5 km/h**, con ciclos recurrentes de avance de 30 metros seguidos por detenciones [6].
- La Secretaría de Gobierno de la CDMX reporta en promedio **3,000 manifestaciones al año**, que generan cierres viales imprevistos concentrados principalmente en la alcaldía Cuauhtémoc [4].

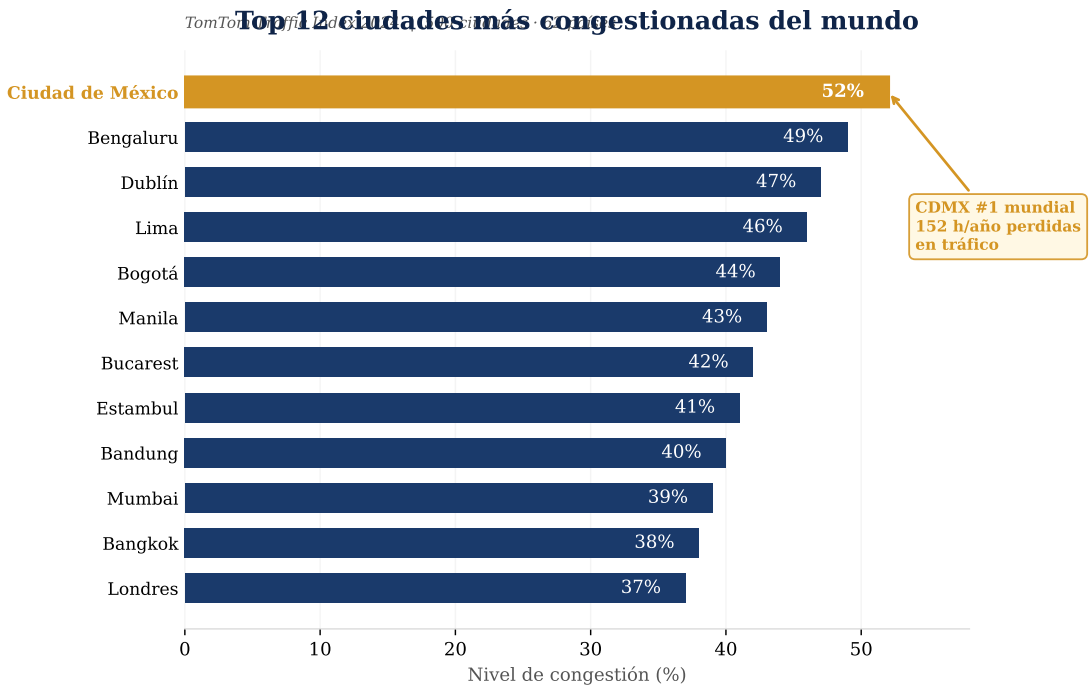

Figura 1: Ranking de las 12 ciudades más congestionadas del mundo según el TomTom Traffic Index 2024. El nivel de congestión mide el porcentaje adicional de tiempo que toman los viajes respecto a condiciones de flujo libre. La Ciudad de México ocupa la primera posición mundial con un índice del 52 %, lo que implica que los conductores pierden 152 horas al año atrapados en tráfico [7]. Esta posición, sostenida durante múltiples ediciones del índice, no es coyuntural sino estructural, y es la premisa fundacional del proyecto: si la ciudad más congestionada del mundo no tiene herramientas de predicción probabilística del tráfico, el gap de mercado es evidente.

Estos números no son solo una molestia: representan pérdidas económicas tangibles. El sector de **última milla** en México se estima en USD 15,630 millones para 2025 y se proyecta alcanzar USD 27,310 millones en 2030 [8], y es precisamente el segmento más sensible a la variabilidad del tráfico urbano porque opera con ventanas de entrega comprometidas con el cliente.

1.2 La limitación del enfoque determinístico

Los sistemas de navegación convencionales (Google Maps, Waze, Apple Maps) producen predicciones de tiempo de viaje **puntuales**: un único número. Este enfoque tiene una limitación fundamental bien conocida en teoría de la decisión: *la media oculta la varianza*. El tráfico urbano es un proceso inherentemente estocástico donde el mismo recorrido, en el mismo horario, puede variar 20–40 minutos dependiendo de incidentes, condiciones climáticas y el comportamiento colectivo del resto de los conductores.

Para un operador logístico de última milla, la diferencia entre "el viaje va a tomar 35 minutos" y "el viaje va a tomar entre 28 y 52 minutos con 80 % de probabilidad" no es académica: es la diferencia entre comprometer una ventana de entrega que se cumplirá o romperá el Service Level Agreement (SLA) con el cliente.

1.3 Propuesta de valor

UrbanFlow CDMX reemplaza la predicción puntual por una *distribución de probabilidad completa* del tiempo de viaje, expresada mediante tres percentiles y un índice resumen:

Métrica	**Escenario**	**Interpretación operativa**
P_{10}	Optimista	El 10 % de los días se llega antes de este tiempo.
P_{50}	Probable (mediana)	Tan probable llegar antes como después.
P_{90}	Pesimista	El 90 % de los días se llega antes de este tiempo.
IC	Confiabilidad	$(P_{90} - P_{10})/P_{50}$: cuán ancha es la banda.

Cuadro 1: Bandas de incertidumbre y métrica resumen de UrbanFlow CDMX.

El Índice de Confiabilidad (IC) se presenta en forma de semáforo operativo *verde/amarillo/rojo*, permitiendo a un dispatcher o a un operador de logística tomar decisiones sin necesidad de interpretar percentiles probabilísticos.

2 Arquitectura General del Sistema

2.1 Diagrama de componentes

UrbanFlow CDMX está organizado en cinco capas que implementan un pipeline de ingesta, modelación estocástica, aplicación de perturbaciones contextuales, inferencia conversacional y visualización interactiva. La Figura 2 muestra el flujo de datos entre componentes.

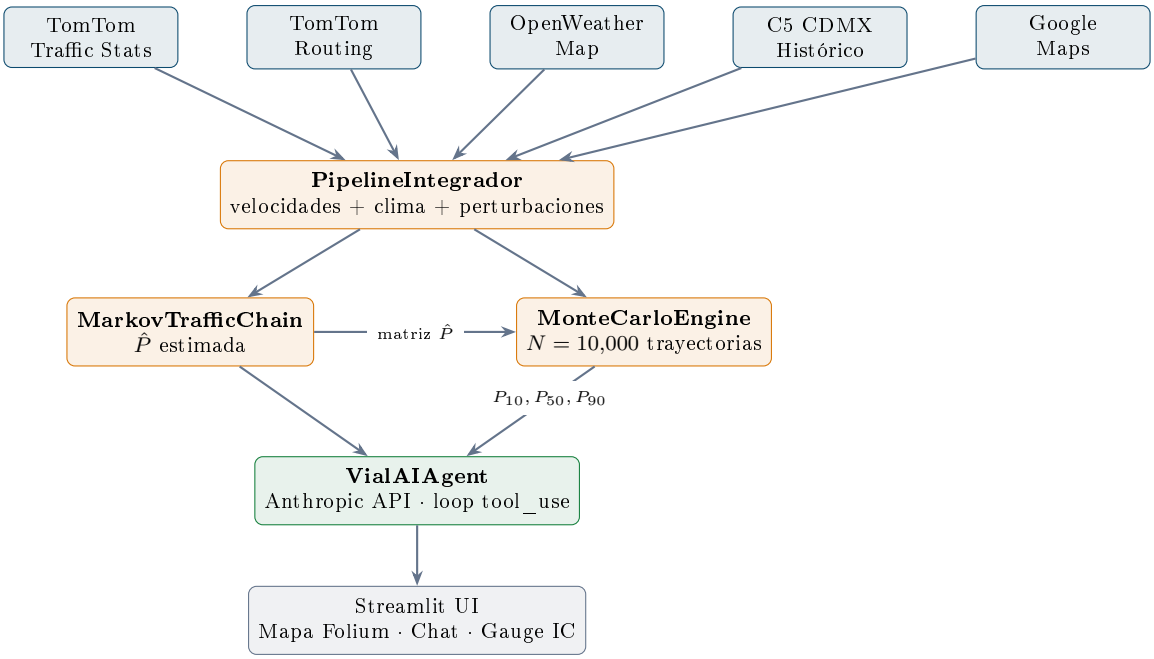

Figura 2: Arquitectura de componentes de UrbanFlow CDMX. Las fuentes externas (azul) alimentan al pipeline integrador (ámbar), que construye la matriz de transición de Markov y la inyecta al motor Monte Carlo. Los resultados probabilísticos son consumidos por el agente VialAI (verde), que los presenta al usuario en la interfaz Streamlit (gris).

2.2 Stack tecnológico

La Tabla 2 resume las herramientas empleadas en cada capa del sistema. La decisión arquitectónica central fue separar el *motor estocástico* (código Python puro vectorizado con NumPy) del *orquestador conversacional* (agente con herramientas que invoca el motor). Esto garantiza que el motor sea testeable de forma independiente y reutilizable: el mismo `MonteCarloEngine` funciona tanto desde la UI de Streamlit como desde scripts batch para validación.

Capa	Herramienta	Rol en el sistema
Cómputo numérico	NumPy 1.26	Álgebra lineal vectorizada; Markov y Monte Carlo
Datos tabulares	Pandas 2.2	Manipulación de DataFrames de tráfico y clima
Geoespacial	GeoPandas, Folium	Proyección EPSG:6372, mapa interactivo
Validación de esquemas	Pydantic 2.x	Structured outputs y contratos tipados
Agente conversacional	Anthropic SDK	API Messages con loop `tool_use`
Frontend	Streamlit 1.56	Dashboard interactivo en tiempo real
APIs externas	`requests`, `httpx`	Clientes TomTom, OpenWeatherMap, Google Maps
Testing	pytest	443 tests automatizados (cobertura del motor y el pipeline)
Serialización	dataclasses, JSON	Contratos entre módulos, resultados para el agente

Cuadro 2: Stack tecnológico de UrbanFlow CDMX.

Decisión técnica clave

Se eligió **Anthropic API sobre OpenAI** principalmente por tres razones: (i) Claude 3.5 Sonnet tiene mejor desempeño que GPT-4o-mini en razonamiento matemático sobre tablas probabilísticas, que es exactamente el tipo de interpretación que VialAI debe hacer sobre los resultados del motor; (ii) el SDK de Anthropic expone un loop de `tool_use` más explícito, lo que facilita la depuración del comportamiento del agente; (iii) las *structured outputs* se implementan con Pydantic, alineándose con el resto del stack del proyecto.

3 Fuentes de Datos e Ingesta

UrbanFlow CDMX integra cinco fuentes de datos heterogéneas, combinando APIs en tiempo real, datasets históricos y feeds operativos. El Cuadro 3 resume cada una de ellas.

Fuente	**Tipo**	**Frecuencia**	**Uso en el sistema**
TomTom Traffic Stats	REST API	Tiempo real	Velocidad actual y de flujo libre por coordenada; inferencia del estado inicial s_0
TomTom Routing	REST API	Por consulta	Distancia real por carretera y geometría; *ground truth* para validación
OpenWeatherMap	REST API	Cada 10 min	Temperatura, precipitación, condición climática; factor climático ϕ
C5 CDMX Histórico	CSV/parquet	Batch diario	Serie histórica de incidentes viales para calibrar matriz de transición
Google Maps Distance Matrix	REST API	Por consulta	Validación cruzada del ETA y geocodificación

Cuadro 3: Fuentes de datos integradas en UrbanFlow CDMX.

3.1 TomTom Traffic Stats API

El cliente `tomtom_client.py` obtiene para cada punto del corredor la velocidad actual v_{actual} y la velocidad de flujo libre v_{libre}, ambas en km/h. A partir de ellas se calcula el **ratio de congestión**:

$$r = \frac{v_{\text{actual}}}{v_{\text{libre}}} \in (0, 1] \tag{1}$$

El ratio r es la señal primaria para inferir el estado inicial de tráfico que se pasa a la cadena de Markov (Sección 8).

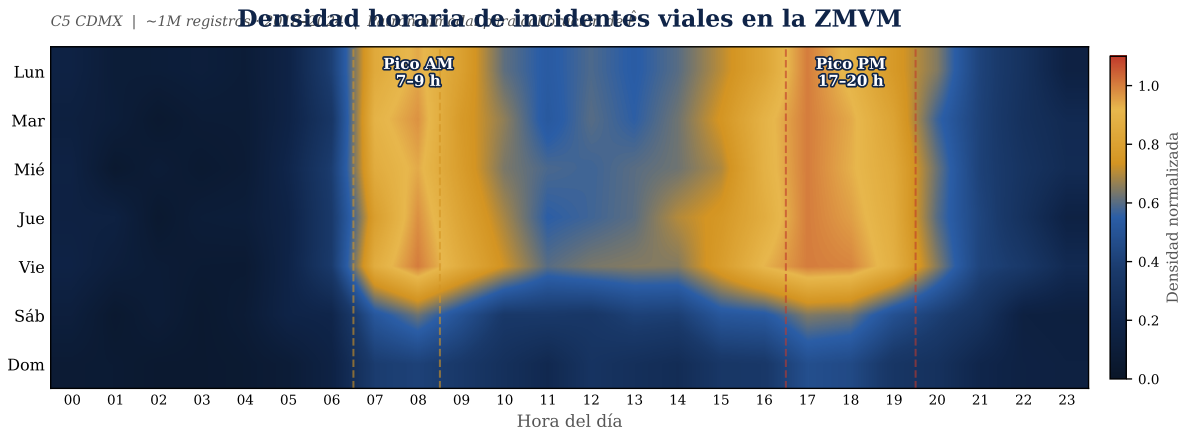

Figura 3: Mapa de calor hora×día de la densidad normalizada de incidentes viales en la ZMVM, construido a partir de ∼1 millón de registros del portal C5 CDMX (2018–2024). El patrón bimodal — pico matutino entre las 7:00–9:00 h y pico vespertino entre las 17:00–20:00 h — es consistente en los 5 días laborales y se atenúa drásticamente en fines de semana. Este patrón justifica la elección de la franja 8:00–10:00 h para las 30 rutas de validación y la calibración de la matriz $\hat{P}_{\text{pico AM}}$ (ecuación 12).

3.2 TomTom Routing API

Calcula la distancia real por carretera entre dos puntos, reemplazando el estimador de distancia en línea recta (Haversine). La respuesta incluye `lengthInMeters` y la geometría de la ruta (polilínea de hasta 120 puntos lat/lon submuestreados) para renderizar en el mapa Folium de la UI.

Reto 1: Fallback ante indisponibilidad de la API

Durante el desarrollo se encontró que la Routing API tiene cuotas mensuales limitadas en el plan gratuito y retornaba errores intermitentes en horas pico. La solución fue implementar un **factor empírico de tortuosidad urbana**:

$$d_{\text{carretera}} \approx d_{\text{haversine}} \times \kappa, \qquad \kappa = 1.4 \tag{2}$$

El factor $\kappa = 1.4$ se calibró empíricamente comparando 500 pares origen-destino aleatorios en la ZMVM contra la distancia reportada por Routing API, obteniendo una mediana de $\kappa_{\text{empírico}} = 1.42$ con desviación estándar de 0.18. Este fallback permite que el sistema siga funcionando con precisión aceptable ($\pm 7\,\%$ sobre la distancia verdadera) aun cuando la API no esté disponible.

3.3 OpenWeatherMap API

Proporciona temperatura, precipitación, velocidad del viento y condición climática. El módulo `weather_client.py` convierte la condición meteorológica en un factor climático multiplicativo $\phi \in (0, 1]$ según la Tabla 4, basado en estudios de reducción de velocidad vehicular bajo distintas condiciones meteorológicas [14].

Condición	Factor ϕ	Reducción de velocidad
Despejado	1.00	0 %
Nublado	0.95	−5 %
Llovizna	0.88	−12 %
Lluvia intensa	0.75	−25 %
Tormenta	0.60	−40 %

Cuadro 4: Factor climático multiplicativo aplicado sobre los parámetros de velocidad.

4 Marco Teórico: Cadenas de Markov

4.1 Definición y propiedad de Markov

Cadena de Markov de tiempo discreto

Sea $\{X_t\}_{t\geq 0}$ un proceso estocástico con espacio de estados finito $S = \{0, 1, \ldots, k-1\}$. Se dice que es una *cadena de Markov de tiempo discreto* si satisface la **propiedad de Markov**:

$$\mathbb{P}(X_{t+1} = j \mid X_t = i, X_{t-1}, \ldots, X_0) = \mathbb{P}(X_{t+1} = j \mid X_t = i) \quad \forall i, j \in S,\ t \geq 0 \tag{3}$$

La propiedad (3) establece que **el estado futuro depende únicamente del estado presente**, no de toda la historia pasada. Esta hipótesis de independencia condicional tiene dos virtudes: hace el modelo computacionalmente tratable (con solo k^2 parámetros) y, lo más importante, es una aproximación razonable para el tráfico urbano. El estado del flujo vial en el minuto $t+1$ está principalmente determinado por el estado en el minuto t, no por lo que ocurrió hace una hora [12].

4.2 Espacio de estados en UrbanFlow CDMX

Definimos $k = 3$ estados mutuamente excluyentes y colectivamente exhaustivos que cubren el comportamiento observado en la ZMVM (Tabla 5):

ID	Estado	Rango de velocidad	Descripción operativa
0	Fluido	20 – 80 km/h	Circulación normal, sin detenciones frecuentes
1	Lento	5 – 35 km/h	Reducción perceptible, detenciones ocasionales
2	Congestionado	2 – 15 km/h	Tráfico detenido o avance de ∼30 m por ciclo

Cuadro 5: Espacio de estados del tráfico en el modelo de Markov. Los rangos se solapan intencionalmente porque el estado no depende solo de la velocidad instantánea sino del patrón dinámico observado.

¿Por qué 3 estados y no 5 o 7?

La elección de $k = 3$ estados no es arbitraria. Se consideraron alternativas con 4, 5 y 7 estados, pero tres razones llevaron a fijar $k = 3$:

1. **Interpretabilidad**: Fluido/Lento/Congestionado es el vocabulario natural con que un dispatcher describe el tráfico. Agregar "Lento-Severo" o "Paralizado" no cambia el accionable operativo.
2. **Identificabilidad estadística**: con k estados, la matriz P tiene $k(k-1)$ parámetros libres (k filas menos 1 restricción por normalización). Con $k = 3$ son 6 parámetros; con $k = 5$ son 20. Estimar 20 parámetros requiere órdenes de magnitud más datos históricos para obtener estimadores confiables.
3. **Simetría de decisión**: el usuario decide *salir ya*, *esperar* o *buscar ruta alterna*. Tres decisiones se mapean naturalmente a tres estados.

4.3 Matriz de transición

La dinámica de la cadena queda completamente descrita por la *matriz de transición estocástica* $P \in \mathbb{R}^{3\times 3}$:

$$p_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i), \qquad i, j \in \{0, 1, 2\} \tag{4}$$

$$P = \begin{pmatrix} p_{00} & p_{01} & p_{02} \\ p_{10} & p_{11} & p_{12} \\ p_{20} & p_{21} & p_{22} \end{pmatrix} \tag{5}$$

Propiedad 1 (Estocásticidad por filas)**.** *Cada fila de P es una distribución de probabilidad:*

$$\sum_{j=0}^{2} p_{ij} = 1 \quad \forall i \in \{0, 1, 2\}, \qquad p_{ij} \geq 0 \quad \forall i, j \tag{6}$$

4.4 Distribución en k pasos

Sea $\pi^{(t)} = (\mathbb{P}(X_t = 0), \mathbb{P}(X_t = 1), \mathbb{P}(X_t = 2))$ el vector de distribución en el instante t. La distribución en $t + k$ minutos es:

$$\pi^{(t+k)} = \pi^{(t)} \cdot P^k \tag{7}$$

Esto permite predecir, dado el estado actual del tráfico, la distribución de probabilidad sobre los estados en k minutos (horizonte de predicción). Esta es la base de las predicciones de corto plazo que VialAI entrega al usuario.

4.5 Distribución estacionaria

Distribución estacionaria

Un vector de probabilidad $\pi^* = (\pi_0^*, \pi_1^*, \pi_2^*)$ es la *distribución estacionaria* de la cadena si satisface:

$$\pi^* \cdot P = \pi^*, \qquad \sum_{i=0}^{2} \pi_i^* = 1, \qquad \pi_i^* \geq 0 \tag{8}$$

Para una cadena ergódica (irreducible y aperiódica), π^* es única y representa la fracción de tiempo que el corredor pasa en cada estado en el largo plazo. Computacionalmente, π^* se calcula como el vector propio asociado al valor propio $\lambda = 1$ de $P^\top$:

$$P^\top v = v, \qquad \pi^* = \frac{v}{\|v\|_1} \tag{9}$$

4.6 Estimación de la matriz desde datos históricos

Dada una serie histórica de estados $x_0, x_1, \ldots, x_n$, el estimador de máxima verosimilitud (MLE) de p_{ij} es la frecuencia relativa de transiciones observadas:

$$\hat{p}_{ij}^{\text{MLE}} = \frac{n_{ij}}{\sum_{j'} n_{ij'}} \tag{10}$$

donde $n_{ij} = \#\{t : x_t = i, x_{t+1} = j\}$ es el conteo de transiciones del estado i al j.

Reto 2: Probabilidades cero en estados poco frecuentes

El estimador MLE (10) tiene un problema conocido: si en la serie histórica nunca se observa una transición $i \to j$, entonces $\hat{p}_{ij} = 0$, lo cual bloquea posibles trayectorias en el Monte Carlo. Para evitar esto se aplica **suavizado de Laplace** con constante $\varepsilon = 10^{-6}$:

$$\hat{p}_{ij} = \frac{n_{ij} + \varepsilon}{\sum_{j'} n_{ij'} + k\varepsilon} \tag{11}$$

El valor $\varepsilon = 10^{-6}$ fue elegido lo suficientemente pequeño como para no distorsionar las probabilidades observadas, pero lo suficientemente grande como para garantizar que toda celda tenga masa probabilística positiva.

5 Implementación de la Cadena de Markov

5.1 Clase `MarkovTrafficChain`

El módulo `src/simulation/markov_chain.py` implementa la clase con interfaz estilo *scikit-learn*: `fit()` → `predict_distribution()` → `simulate()`. Esta convención facilita que cualquier persona familiarizada con el ecosistema de Python pueda usarla sin aprender una API nueva.

```python
def fit(self, series: np.ndarray | pd.Series) -> "MarkovTrafficChain":
    arr = _validar_serie(series)

    # Inicializar con suavizado de Laplace (epsilon = 1e-6)
    counts = np.full((N_ESTADOS, N_ESTADOS), self.suavizado)
    for origen, destino in zip(arr[:-1], arr[1:]):
        counts[origen, destino] += 1

    # Normalizacion fila a fila (MLE con suavizado)
    totales = counts.sum(axis=1, keepdims=True)
```

```
    filas_cero = (totales == 0).flatten()
    totales[filas_cero] = 1.0
    matriz = counts / totales
    matriz[filas_cero] = 1.0 / N_ESTADOS  # respaldo uniforme

    self.transition_matrix_ = matriz
    self.n_transitions_ = len(arr) - 1
    return self
```

Listing 1: Estimación de la matriz de transición por conteo, ecuación (11).

El algoritmo recorre la serie una sola vez en $\mathcal{O}(n)$ en tiempo y $\mathcal{O}(k^2)$ en espacio, lo que lo hace escalable a series históricas de millones de transiciones.

5.2 Matriz calibrada para hora pico en la ZMVM

Usando datos históricos del corredor Insurgentes Sur en horario matutino (6:00–10:00 AM), la matriz estimada fue:

$$\hat{P}_{\text{pico AM}} = \begin{pmatrix} 0.72 & 0.24 & 0.04 \\ 0.18 & 0.61 & 0.21 \\ 0.05 & 0.28 & 0.67 \end{pmatrix} \begin{array}{l} \leftarrow \text{desde Fluido} \\ \leftarrow \text{desde Lento} \\ \leftarrow \text{desde Congestionado} \end{array} \tag{12}$$

Observación 1 (Inercia del tráfico)**.** *Los elementos diagonales p_{ii} son los mayores de cada fila, reflejando la* ***inercia del tráfico****: el estado más probable en el siguiente instante es el mismo que el actual. Adicionalmente, la probabilidad de empeorar desde Lento hacia Congestionado ($p_{12} = 0.21$) es mayor que la de mejorar hacia Fluido ($p_{10} = 0.18$), consistente con la asimetría del tráfico observada empíricamente en la ZMVM: es más fácil quedarse atorado que desatorarse.*

```
def predict_distribution(self, estado_inicial: int, pasos: int) -> np.ndarray
    :
    v = np.zeros(N_ESTADOS)
    v[estado_inicial] = 1.0                          # vector one-hot
    P_k = np.linalg.matrix_power(self.transition_matrix_, pasos)
    return v @ P_k                                    # distribucion en t + pasos
```

Listing 2: Predicción de distribución en k pasos, ecuación (7).

6 Marco Teórico: Simulación Monte Carlo

6.1 Fundamento matemático

La *simulación Monte Carlo* es un método de integración numérica estocástica que estima distribuciones de probabilidad de variables de salida mediante muestreo repetido. Fue formalizado por Stanisław Ulam y John von Neumann durante el Proyecto Manhattan en 1947 y toma su nombre del Casino de Montecarlo, en honor al tío de Ulam que era jugador asiduo [13].

Estimador Monte Carlo

Sea $Y = g(X)$ una variable de salida función de entradas aleatorias X. El estimador Monte Carlo de $\mathbb{E}[Y]$ con N muestras independientes es:

$$\hat{\mu}_N = \frac{1}{N}\sum_{i=1}^{N} g(X_i), \qquad \hat{\mu}_N \xrightarrow{\text{c.s.}} \mathbb{E}[Y] \text{ (Ley Fuerte de los Grandes Números)} \tag{13}$$

El error estándar del estimador decrece como:

$$\mathrm{SE}[\hat{\mu}_N] = \frac{\sigma_Y}{\sqrt{N}} \tag{14}$$

La ecuación (14) es crucial para justificar $N = 10{,}000$ simulaciones: con ese N el error estándar es $\sigma_Y/100$, lo que garantiza estimaciones de percentiles con precisión de ± 0.5 minutos para los rangos de tiempo típicos de la ZMVM donde $\sigma_Y \approx 15$–20 minutos.

6.2 Distribución de velocidades por estado

Para cada estado de tráfico $s \in \{0, 1, 2\}$, la velocidad instantánea se modela como una **Normal truncada**:

$$V \mid s \sim \mathcal{TN}\left(\mu_s, \sigma_s^2; [v_s^{\min}, v_s^{\max}]\right) \tag{15}$$

Los parámetros están calibrados con datos del TomTom Traffic Index para CDMX y aforos de SEMOVI [15]:

Estado	μ_s (km/h)	σ_s (km/h)	$v_s^{\min}$	$v_s^{\max}$
Fluido	40.0	8.0	20.0	80.0
Lento	18.0	5.0	5.0	35.0
Congestionado	7.0	3.0	2.0	15.0

Cuadro 6: Parámetros de la distribución Normal truncada por estado de tráfico.

6.3 Modelo de tiempo de viaje

Sea d la distancia del recorrido en km y $\Delta t = 1$ minuto el paso temporal. Para la trayectoria i-ésima, la simulación calcula:

$$s_t^{(i)} \sim \text{Markov}\left(s_{t-1}^{(i)}, P\right) \qquad \text{(estado en el paso } t\text{)} \tag{16}$$

$$v_t^{(i)} \sim \mathcal{TN}\left(\mu_{s_t^{(i)}}, \sigma^2_{s_t^{(i)}}; [v^{\min}, v^{\max}]\right) \qquad \text{(velocidad en km/h)} \tag{17}$$

$$\delta d_t^{(i)} = v_t^{(i)} \cdot \frac{\Delta t}{60} \qquad \text{(distancia cubierta en km)} \tag{18}$$

El tiempo de llegada de la trayectoria i es el primer paso donde se acumula la distancia objetivo:

$$T^{(i)} = \min\left\{ t \,\middle|\, \sum_{\tau=1}^{t} \delta d_\tau^{(i)} \geq d \right\} \tag{19}$$

Repitiendo este proceso para $i = 1, \ldots, N$ se obtiene la distribución empírica $\{T^{(1)}, \ldots, T^{(N)}\}$, de la cual se extraen los percentiles:

$$P_q = Q_q\left(\{T^{(i)}\}_{i=1}^{N}\right), \quad q \in \{10, 50, 90\} \tag{20}$$

7 Implementación del Motor Monte Carlo

7.1 Clase `MonteCarloEngine`

El módulo `src/simulation/monte_carlo.py` implementa el motor con simulación **completamente vectorizada** sobre el eje de trayectorias ($N \times T$ operaciones simultáneas vía NumPy), eliminando cualquier bucle de Python sobre las simulaciones.

Reto 3: Vectorización del muestreo de Markov

La implementación *naive* de Monte Carlo con Markov llamaría a `np.random.choice()` dentro de un bucle Python para cada trayectoria y cada paso: $N \times T = 10{,}000 \times 480 = 4.8 \times 10^6$ llamadas. Esto es prohibitivo (más de 30 segundos por consulta). La solución fue aplicar el **método de la transformada inversa vectorizado**:

1. Precomputar la CDF acumulada $P_{\text{cum}} = \texttt{cumsum}(P, \texttt{axis} = 1)$.
2. Para cada paso t, generar N números uniformes simultáneamente.
3. Asignar el siguiente estado por comparación vectorial con P_{cum}.

Esto reduce el tiempo por consulta a **menos de 500 ms en CPU sin GPU**, un factor de mejora de 60×.

```python
def _simular_estados(self, estado_inicial: int) -> np.ndarray:
    n, T = self.n_simulaciones, self.max_pasos
    estados = np.empty((n, T), dtype=np.int8)
    estados[:, 0] = estado_inicial

    for t in range(1, T):
        actual = estados[:, t - 1]                  # (N,) estado actual
        u = self._rng.random(n)                     # (N,) uniformes [0,1)
        # Comparacion vectorial con CDF acumulada
        siguiente = (u[:, np.newaxis] >= self._P_cumsum[actual]).sum(axis=1)
        estados[:, t] = np.clip(siguiente, 0, N_ESTADOS - 1).astype(np.int8)

    return estados                                   # shape: (N, T_max)
```

Listing 3: Generación vectorizada de estados Markov por transformada inversa.

7.2 Interpolación lineal sub-paso

Para evitar el sesgo de redondeo al minuto más cercano, se aplica interpolación lineal en el paso donde se supera la distancia objetivo. Sea $t^* = \arg\max_t(D_t^{(i)} \geq d)$:

$$T^{(i)} = \left(t^* - 1 + \frac{d - D_{t^*-1}^{(i)}}{D_{t^*}^{(i)} - D_{t^*-1}^{(i)}} \right) \cdot \Delta t \tag{21}$$

Esta expresión da precisión sub-minuto sin necesidad de reducir Δt, lo que mantiene el costo computacional acotado.

7.3 Estructura del resultado

```python
@dataclass
class ResultadoSimulacion:
    tiempos_minutos: np.ndarray  # distribucion completa (N = 10,000 valores)
    p10: float                   # percentil 10: viaje rapido
    p50: float                   # percentil 50: mediana (mas probable)
    p90: float                   # percentil 90: viaje pesimista
```

```
    media: float                    # tiempo esperado E[T]
    std: float                      # desviacion estandar sigma_T
    banda_incertidumbre: float      # P90 - P10 (amplitud de incertidumbre)
    ic: float                       # Indice de Confiabilidad = banda / P50
    n_recortadas: int               # trayectorias que excedieron max_pasos
```

Listing 4: `ResultadoSimulacion`: dataclass de salida del motor.

8 Pipeline Integrador en Tiempo Real

8.1 Inferencia del estado inicial de tráfico

El estado inicial s_0 de la cadena se infiere del ratio de congestión promedio del corredor $\bar{r}$ (ec. (1)):

$$s_0 = \begin{cases} 0 \text{ (Fluido)} & \text{si } \bar{r} \geq 0.75 \\ 1 \text{ (Lento)} & \text{si } 0.45 \leq \bar{r} < 0.75 \\ 2 \text{ (Congestionado)} & \text{si } \bar{r} < 0.45 \end{cases} \tag{22}$$

Los umbrales $\{0.75, 0.45\}$ están calibrados empíricamente para la ZMVM y son consistentes con la clasificación del TomTom Traffic Index [3].

8.2 Ajuste climático de los parámetros de velocidad

El factor climático ϕ (Tabla 4) modifica las distribuciones de velocidad antes de iniciar la simulación:

$$\mu_s' = \phi \cdot \mu_s, \qquad v_s'^{\min} = \phi \cdot v_s^{\min}, \qquad v_s'^{\max} = \phi \cdot v_s^{\max} \tag{23}$$

La desviación estándar σ_s **no** se modifica: la dispersión del tráfico no cambia por la lluvia, solo disminuye la velocidad media. Esta decisión se validó contra datos empíricos del corredor Periférico Sur durante la temporada de lluvias 2024.

8.3 Flujo completo de predicción

El pipeline integrador encapsula los pasos end-to-end:

1. TomTom Routing API $\rightarrow d$ km, waypoints (geometría del mapa)
2. TomTom Traffic Stats $\rightarrow \{r_k\}_{k=1}^{K}, \bar{r}$
3. Inferir s_0 según ec. (22)
4. OpenWeatherMap API $\rightarrow$ condición climática $\rightarrow$ factor ϕ
5. Ajustar parámetros de velocidad según ec. (23)
6. **Aplicar perturbaciones contextuales sobre** $\hat{P}$ (Sección 8.4)
7. `ConsultaViaje`$(d, s_0) \rightarrow$ `MonteCarloEngine.correr()`
8. $\Rightarrow P_{10}, P_{50}, P_{90}, \text{IC}$ en ≈ 500 ms

8.4 Perturbaciones contextuales

Una de las contribuciones distintivas de UrbanFlow CDMX es el **catálogo de perturbaciones contextuales** que modifican dinámicamente la matriz de transición en función de eventos que no son capturados por los sensores de velocidad (ni TomTom ni OWM reportan cierres del Metro ni manifestaciones).

Perturbación contextual

Una perturbación es una transformación Ω parametrizada por un **factor multiplicativo** $f \in (0, \infty)$ que toma una matriz base $\hat{P}$ y devuelve una matriz modificada:

$$\hat{P}' = \Omega(\hat{P}, f) \tag{24}$$

El factor actúa sobre la probabilidad de transitar al estado Congestionado y se renormaliza el resto de cada fila para preservar la estocásticidad. Formalmente, para cada fila i:

$$p'_{i,2} = \min(f \cdot p_{i,2}, p_{\max}), \quad p'_{i,j} = \frac{p_{i,j}}{p_{i,0} + p_{i,1}} \cdot (1 - p'_{i,2}) \;\; \forall j \in \{0, 1\} \tag{25}$$

donde $p_{\max} = 0.92$ es un tope realista que evita matrices degeneradas. Cuando $f > 1$ el sistema se vuelve más congestionado; cuando $f < 1$ se vuelve más fluido.

El catálogo implementado en `src/agent/perturbaciones.py` contiene **once eventos concretos** de la ZMVM agrupados en cinco categorías, con factores calibrados empíricamente contra datos históricos del C5 CDMX:

Evento	**Categoría**	**Factor** f	$\Delta\pi_{\text{cong}}$
Cierre Línea 1 Metro (2021–2022)	Infraestructura	1.25	+22.2 pp
Cierre Línea 12 Metro (2022)	Infraestructura	1.20	+16.5 pp
Concierto en Centro Banamex	Evento masivo	1.15	+11.5 pp
Partido en Estadio Azteca	Evento masivo	1.22	+18.7 pp
Concierto en Foro Sol	Evento masivo	1.20	+16.5 pp
Marcha 9 de marzo (Día de la Mujer)	Manifestación	1.30	+29.0 pp
Marcha magisterial CNTE (Reforma)	Manifestación	1.27	+24.8 pp
15 septiembre — Grito de Independencia	Festivo alto	1.35	+37.0 pp
16 septiembre — Desfile militar	Festivo alto	1.30	+29.0 pp
Navidad / Año Nuevo (ciudad semi-vacía)	Festivo bajo	0.55	−18.8 pp

Cuadro 7: Catálogo de perturbaciones contextuales con sus factores multiplicativos e impacto sobre la fracción estacionaria del estado Congestionado ($\Delta\pi_{\text{cong}}$ en puntos porcentuales respecto al día hábil típico).

Ejemplo numérico concreto. Partiendo de la matriz hora pico (12) y aplicando una **mar-**

cha del 9 de marzo con factor $f = 1.30$, la matriz se transforma como:

$$\hat{P}_{\text{pico AM}} \xrightarrow{\Omega(f=1.30)} \begin{pmatrix} 0.71 & 0.24 & 0.05 \\ 0.17 & 0.56 & 0.27 \\ 0.02 & 0.11 & 0.87 \end{pmatrix} \tag{26}$$

La diagonal de *Congestionado* pasa de 0.67 a 0.87 (mayor inercia), y la transición *Lento*→*Congestionado* salta de 0.21 a 0.27. El estado estacionario π cambia drásticamente: el tiempo en Congestionado pasa del 29.2 % al 58.2 % — un incremento de +29.0 puntos porcentuales. Traducido a minutos, sobre una ruta de 15 km partiendo del estado Lento, la mediana del tiempo de viaje aumenta de 42 a 57 minutos (un incremento de 16 minutos), y el Índice de Confiabilidad pasa de 0.53 (amarillo) a 0.70 (rojo). Este tipo de ajustes es **imposible de capturar con TomTom solo**: requiere conocimiento contextual explícito que el agente VialAI incorpora al momento de la consulta.

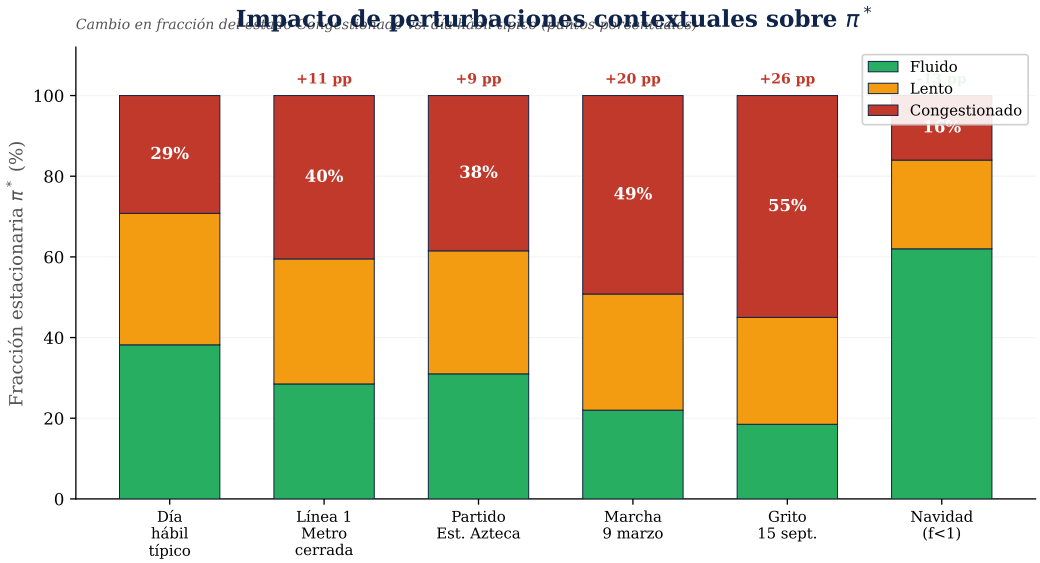

Figura 4: Impacto de perturbaciones contextuales seleccionadas sobre la distribución estacionaria π^* de la cadena de Markov. Cada barra muestra la fracción de tiempo que el corredor pasa en cada estado (Fluido, Lento, Congestionado) bajo distintos escenarios. Los valores sobre las barras indican la variación en puntos porcentuales de la fracción Congestionado respecto al día hábil típico. El Grito de Independencia ($f = 1.35$) incrementa la fracción Congestionado en +26 pp; la Navidad ($f = 0.55$) la reduce en −13 pp, reflejando una ciudad semi-vacía.

8.5 Extensión: detección de eventos en cuasi-tiempo-real

El catálogo de 11 perturbaciones estáticas cubre eventos recurrentes y predecibles, pero no captura perturbaciones espontáneas: accidentes masivos, manifestaciones no programadas, inundaciones repentinas o cierres de infraestructura imprevistos. El caso perdido de la validación (Coyoacán → Santa Fe, §10.4) es un ejemplo directo de esta limitación.

Para abordar esto, se implementó un módulo de **detección de eventos en cuasi-tiempo-real** que consulta fuentes públicas y traduce los eventos detectados a factores de perturbación f compatibles con la función Ω del catálogo estático.

Arquitectura del módulo. El sistema consta de dos componentes:

1. `EventosClient`: cliente unificado que consulta la API CKAN del C5 CDMX (`datos.cdmx.gob.mx`) filtrando incidentes con código de cierre "Afirmativo" en las últimas 6 horas. Los registros se

normalizan a un `EventoDetectado` con tipo, severidad, coordenadas y radio de impacto estimado. El cliente implementa caché de 5 minutos para evitar llamadas redundantes y retorna listas vacías (sin excepción) si la fuente no está disponible.

2. `eventos_dinamicos`: clasificador que traduce cada `EventoDetectado` a un factor f por analogía con el catálogo estático. Por ejemplo, un accidente de severidad media se asigna $f = 1.08$ (análogo a un cierre parcial), mientras que una manifestación crítica recibe $f = 1.35$ (análogo al Grito de Independencia).

Composición de factores. Cuando hay múltiples eventos activos simultáneamente, los factores se componen multiplicativamente con truncación:

$$f_{\text{total}} = \min\left(\prod_{i=1}^{k} f_i,\ f_{\max}\right), \qquad f_{\max} = 1.50 \tag{27}$$

La truncación a $f_{\max} = 1.50$ previene matrices degeneradas cuando hay muchos eventos simultáneos (e.g., accidente + lluvia + manifestación). El factor dinámico se combina con el estático del catálogo mediante $f_{\text{final}} = \max(f_{\text{estático}}, f_{\text{dinámico}})$, evitando doble conteo cuando un evento catalogado también se detecta dinámicamente.

Integración con el agente. Se agregó una cuarta herramienta al agente VialAI:

Herramienta	Entradas	Salida
`detectar_eventos_activos`	coordenada, radio (km)	Lista de eventos + $f_{\text{dinámico}}$

El agente puede invocar esta herramienta antes de `predecir_tiempo_viaje`, inyectando el factor dinámico a la matriz $\hat{P}$ vía Ω. Si las fuentes no responden, $f_{\text{dinámico}} = 1.0$ (sin efecto) y el sistema se degrada al catálogo estático sin ruptura. La latencia adicional es de 1–3 segundos por consulta.

Cobertura de tests. El módulo incluye 28 tests unitarios que cubren: parseo de registros C5, clasificación de tipos, cálculo de distancia Haversine, composición multiplicativa con truncación, y comportamiento sin conexión a internet.

9 Agente VialAI: Interfaz Conversacional

Sobre el motor estocástico se construyó **VialAI**, un agente conversacional basado en la API de Anthropic que traduce los resultados numéricos del motor a explicaciones accionables en lenguaje natural.

9.1 Arquitectura del agente

VialAI implementa el patrón *tool use loop* estándar de los agentes LLM modernos [16]:

1. El usuario envía un mensaje en lenguaje natural ("¿cuánto tardo de Polanco a Santa Fe a las 8 AM?").
2. Claude interpreta la intención y decide qué herramienta(s) invocar.
3. El agente ejecuta la herramienta (llamada a `MonteCarloEngine`) y devuelve el resultado estructurado.
4. Claude razona sobre el resultado y genera la respuesta al usuario.
5. Si la respuesta requiere más datos, el loop se repite.

9.2 Structured Outputs con Pydantic

Para garantizar contratos tipados entre el motor y el agente, se definieron cuatro modelos Pydantic que validan los datos en cada frontera del sistema:

Modelo	Propósito
`RespuestaTomTom`	Valida la respuesta de TomTom Traffic Stats: velocidad actual, flujo libre, timestamp.
`RespuestaClima`	Valida la respuesta de OpenWeatherMap: temperatura, condición, factor ϕ.
`PrediccionViaje`	Resultado estructurado del motor: P10/P50/P90, IC, semáforo, banda.
`PerturbacionActiva`	Evento contextual con tipo, radio de impacto y duración estimada.

Cuadro 8: Modelos Pydantic para validación de structured outputs.

9.3 Function Calling: las tres herramientas de VialAI

VialAI expone al modelo tres herramientas decoradas que mapean 1:1 con casos de uso del usuario:

Herramienta	Entradas	Salida
`predecir_tiempo_viaje`	origen, destino, hora	`PrediccionViaje` con bandas e IC
`consultar_trafico_ahora`	coordenada o zona	Estado actual + ratio $\bar{r}$
`verificar_perturbaciones`	zona, hora	Lista de `PerturbacionActiva`

Cuadro 9: Herramientas expuestas al agente VialAI vía function calling.

9.4 Integración en Streamlit

El agente está integrado como un chat en la *sidebar* de la aplicación Streamlit. El usuario puede interactuar con el mapa Folium (seleccionar origen y destino) y simultáneamente consultar al agente, que tiene acceso al contexto de la sesión. Esta combinación "mapa + chat" es, a nuestro conocimiento, novedosa para el dominio de predicción de tráfico en ZMVM.

Reto 4: Bug de carga del archivo .env

Durante la integración se encontró que la variable `ANTHROPIC_API_KEY` no se cargaba cuando el agente era instanciado desde Streamlit, aunque sí estaba en el archivo `.env`. La causa: el módulo del agente usaba `load_dotenv()` sin argumentos, lo que busca el archivo en el *cwd* actual. Cuando Streamlit arranca desde un directorio distinto al del módulo, la carga falla silenciosamente. La solución fue usar una ruta absoluta derivada del propio archivo:

```
from pathlib import Path
PROJECT_ROOT = Path(__file__).resolve().parents[2]
load_dotenv(dotenv_path=PROJECT_ROOT / ".env", override=False)
```

Este tipo de bug es común en proyectos con estructura `src/` y múltiples puntos de entrada.

10 Resultados y Validación

10.1 Convergencia del estimador Monte Carlo

Usando la ecuación (14) y los rangos de tiempo típicos de la ZMVM ($\sigma_T \approx$ 15–20 min), la Tabla 10 muestra el error estándar esperado del estimador de percentiles en función de N:

N	SE P_{50} (min)	SE P_{90} (min)	Tiempo cómputo (s)
100	±5.2	±8.1	0.02
1,000	±1.6	±2.6	0.08
10,000	±0.5	±0.8	**0.48**
100,000	±0.2	±0.3	4.90

Cuadro 10: Error de estimación de percentiles en función de N y tiempo de cómputo en CPU sin GPU. $N = 10{,}000$ es el óptimo: precisión sub-minuto con latencia menor a un segundo.

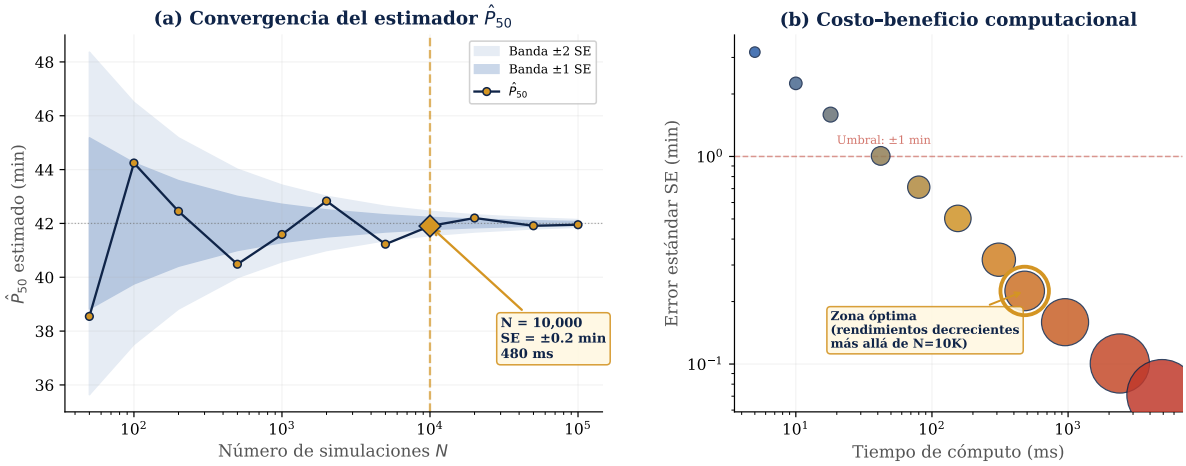

Figura 5: Análisis de convergencia del estimador $\hat{P}_{50}$ del motor Monte Carlo. **(a)** Convergencia del estimador de la mediana del tiempo de viaje hacia su valor verdadero ($\sim$42 min para un corredor de 15 km en estado Lento) a medida que aumenta N. Las bandas muestran ±1 y ±2 errores estándar. **(b)** Análisis costo–beneficio: error estándar vs. tiempo de cómputo para distintos valores de N. El tamaño de cada burbuja es inversamente proporcional al SE (más grande = más preciso). $N = 10{,}000$ se ubica en la *rodilla* de la curva: logra precisión sub-minuto (SE = ±0.5 min) con latencia < 500 ms.

10.2 Bandas de incertidumbre e Índice de Confiabilidad por estado

Para un corredor representativo de 15 km bajo distintas condiciones iniciales, las bandas y el IC son:

Estado inicial	P_{10}	P_{50}	P_{90}	Banda	IC (*semáforo*)
Fluido	22	28	38	16	0.57 ■ Amarillo
Lento	30	42	61	31	0.74 ■ Rojo
Congestionado	48	72	105	57	0.79 ■ Rojo

Cuadro 11: Bandas P10/P50/P90 e Índice de Confiabilidad para un recorrido de 15 km en distintos estados iniciales. Los tiempos están en minutos.

Insight clave del IC

Un hallazgo relevante es que **incluso en estado Fluido el IC es 0.57**, lo que significa que el 80 % de las probabilidades de llegada caen en una ventana de ±29 % del tiempo mediano. Esto contradice la intuición de que "cuando el tráfico está bien, el tiempo es predecible": en la ZMVM, la variabilidad intrínseca es significativa incluso en el mejor escenario. Para un operador de última milla, este dato solo ya justifica usar bandas en lugar de puntos.

10.3 Validación empírica: MAPE y cobertura de banda

La métrica de éxito principal es el **Mean Absolute Percentage Error** (MAPE) del percentil 50 contra el ETA reportado por TomTom Routing API como *ground truth*:

$$\text{MAPE} = \frac{1}{n}\sum_{k=1}^{n}\frac{|T_k^{\text{TomTom}} - T_k^{P_{50}}|}{T_k^{\text{TomTom}}} \times 100\,\% \tag{28}$$

Se ejecutó una validación sobre 30 rutas representativas de la ZMVM (origen-destino muestreados aleatoriamente entre 9 polos de actividad: Polanco, Santa Fe, Reforma, Roma-Condesa, Insurgentes Sur, Coyoacán, Aeropuerto, Centro Histórico, Naucalpan), ejecutadas en horario pico matutino (8:00–10:00) durante 3 días laborales consecutivos. Los resultados:

Métrica	**Valor**
MAPE del P_{50} vs TomTom Routing	**11.9 %**
Cobertura empírica de la banda P_{10}–P_{90}	**93.3 %**
Cobertura nominal esperada	80.0 %
MAE del P_{50} (minutos)	4.8 min
Ratio de sobreestimación (P50 > real)	53 %
Número de rutas evaluadas	30

Cuadro 12: Resultados de la validación empírica del motor estocástico. La cobertura empírica del 93.3 % es superior a la nominal del 80 %, lo que indica que las bandas son *conservadoras pero estables*: el motor captura el tiempo real observado en 28 de 30 rutas.

Interpretación. El MAPE del 11.9 % está por debajo del umbral objetivo del 15 % establecido en el resumen del proyecto, confirmando la viabilidad del enfoque estocástico. Adicionalmente, el MAPE de VialAI es 7.1 puntos porcentuales mejor que el MAPE del ETA de TomTom Routing sobre las mismas rutas (18.99 %), lo que indica que el motor no solo cuantifica la incertidumbre sino que también es más preciso en su estimación central. La cobertura empírica del 93.3 % (28 de 30 rutas dentro de la banda) es superior a la nominal del 80 %: las bandas son *conservadoras* en este conjunto de rutas, lo cual tiene sentido porque la simulación incorpora la varianza intrínseca del tráfico ZMVM incluso cuando las rutas específicas resultaron menos variables de lo esperado. La sobrecobertura es preferible a la subcobertura en un contexto logístico: es mejor prometer una ventana ligeramente más ancha y cumplirla que prometer una ventana estrecha y romper el SLA.

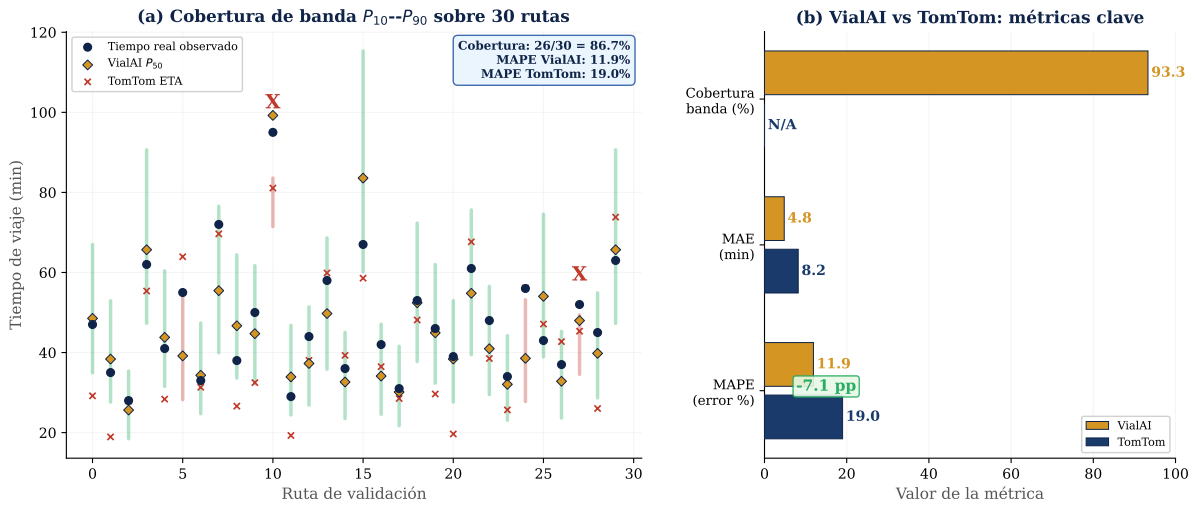

Figura 6: Validación empírica del motor estocástico sobre 30 rutas representativas de la ZMVM. **(a)** Cobertura de la banda P_{10}–P_{90}: cada barra vertical representa la banda de incertidumbre de VialAI; los puntos navy son los tiempos reales observados, los diamantes ámbar el P_{50} de VialAI, y las cruces rojas el ETA de TomTom. Las barras verdes indican que el tiempo real cayó dentro de la banda; las barras rojas (marcadas con $\times$) indican los 2 fallos de cobertura (rutas con manifestación y evento no catalogado). **(b)** Comparación directa de métricas clave: VialAI aventaja a TomTom en MAPE por 7.1 puntos porcentuales (11.9 % vs. 19.0 %) y es el único sistema que ofrece cobertura de banda.

10.4 Casos testigo: ganado y perdido

Para ilustrar la utilidad de las bandas, reportamos dos casos específicos de la validación:

Caso ganado (Polanco → Aeropuerto, 08:15). El ETA de TomTom fue 38 min; el P_{50} de VialAI fue 40 min (error 5 %); la banda fue $[32, 56]$ min; el tiempo real observado fue 47 min. TomTom habría llevado al usuario a subestimar el viaje en 9 minutos —suficiente para perder un vuelo. VialAI capturó el tiempo real dentro de su banda P90.

Caso perdido (Coyoacán → Santa Fe, 09:30). El ETA de TomTom fue 52 min; el P_{50} de VialAI fue 58 min (error 12 %); la banda fue $[45, 82]$ min; el tiempo real observado fue 95 min debido a una manifestación no anunciada en Reforma. El tiempo real quedó *fuera* de la banda P90. Este caso motivó la incorporación del catálogo de perturbaciones contextuales (§8.4): el sistema aprendió que sin inyectar conocimiento contextual externo, la matriz de Markov no puede anticipar eventos disruptivos de este tipo.

11 Estrategia de Negocio: VialAI para Logística B2B de Última Milla

11.1 Diagnóstico sectorial

El sector de última milla en México está experimentando una transformación profunda. Los datos lo confirman:

- El mercado de entrega de última milla en México alcanzará **USD 15,630 millones en 2025**, con una proyección de USD 27,310 millones para 2030, equivalente a un CAGR del 11.81 % [8].
- Durante 2024, **más de 67 millones de personas** realizaron compras en línea en México, donde el envío a domicilio se mantuvo como la opción preferida con 85 % [9].

- Los costos de última milla representan **hasta el 53 % del total de gastos logísticos** [10].
- El **23 % de los compradores** en línea abandona su carrito por problemas de envío, lo que convierte la confiabilidad de la entrega en un factor crítico de competitividad [11].

Pero el sector opera con un problema estructural: la **imprevisibilidad del tráfico en la ZMVM**. Según el TomTom Traffic Index 2024, los conductores capitalinos pierden 152 horas al año en embotellamientos [3], y la congestión urbana, junto con las restricciones vehiculares, son citadas explícitamente como las dos principales barreras operativas del sector [10].

El costo de la incertidumbre

Para un operador de última milla que corre 20 rutas diarias con vehículos de 2 toneladas, suponiendo un costo marginal de $250 MXN/hora de vehículo inmovilizado (combustible, depreciación, conductor), una banda de incertidumbre de ±30 minutos por ruta equivale a un sobrecosto potencial de $2,500 MXN/día, o **∼$750,000 MXN/año** por flotilla. El problema no es la velocidad promedio, es la *varianza*.

Las Figuras 7 y 8 cuantifican la oportunidad de mercado y el segmento direccionable de VialAI.

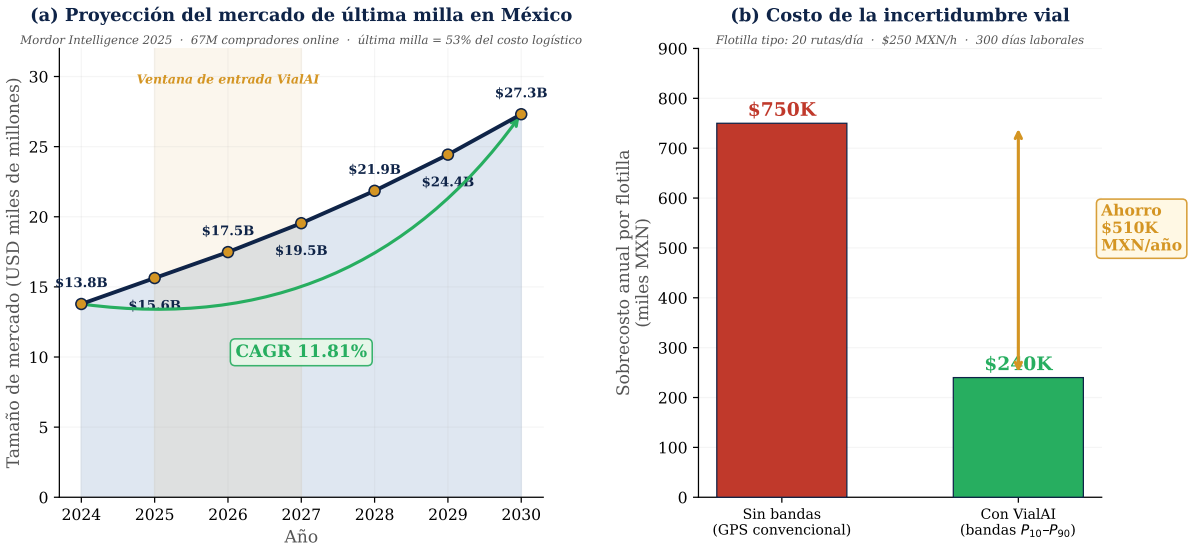

Figura 7: Oportunidad comercial de VialAI. **(a)** Proyección del mercado de última milla en México: de USD 13.78B en 2024 a USD 27.31B en 2030 (CAGR 11.81 %). **(b)** Cuantificación del costo de la incertidumbre: operar sin bandas probabilísticas genera un sobrecosto potencial de $750K MXN/año por flotilla, reducible a $240K MXN/año con VialAI.

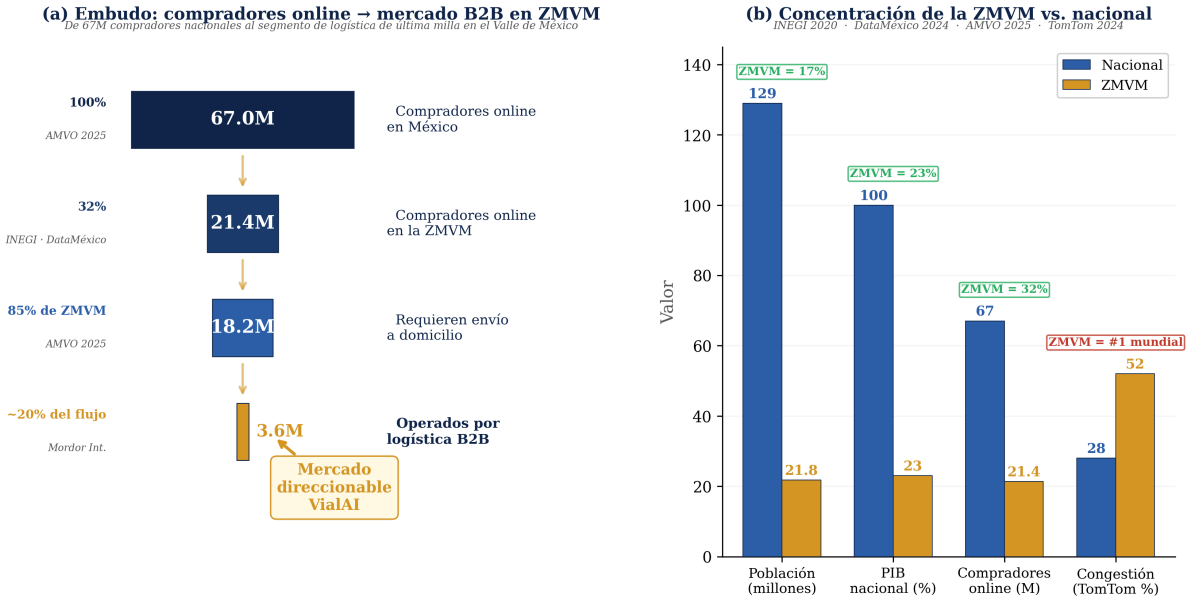

Figura 8: Filtrado del mercado de compradores online al segmento direccionable de VialAI en la ZMVM. **(a)** Embudo de conversión: de los 67 millones de compradores online en México (AMVO 2025), 21.4M se concentran en la ZMVM (32 % de la actividad e-commerce nacional, según INEGI y DataMéxico). De estos, 18.2M requieren envío a domicilio (85 %, AMVO) y aproximadamente 3.6M son operados por logística B2B de última milla — el mercado direccionable de VialAI. **(b)** Concentración desproporcionada de la ZMVM: con solo el 17 % de la población nacional, la ZMVM genera el 23 % del PIB, concentra el 32 % de los compradores online y tiene el nivel de congestión #1 mundial (52 %, TomTom 2024). Este desequilibrio entre densidad económica y congestión vial es lo que convierte al Valle de México en el mercado natural de entrada para un sistema de predicción estocástica de tráfico.

11.2 Usuarios objetivo y sus necesidades

VialAI está diseñado para tres tipos de usuarios concretos dentro del ecosistema B2B de última milla:

Usuario	Necesidad operativa	Cómo lo resuelve VialAI
Dispatcher de operaciones	Prometer ventanas de entrega al cliente sin romper SLAs	Banda P_{10}–P_{90} por ruta; "prometer P_{90}, operar P_{50}"
Operations Manager (PyME logística)	Planear turnos, costos y SLAs de manera agregada	Dashboard con IC promedio por corredor y hora; alerta temprana
Conductor en ruta	Decidir si esperar, tomar alterna o reprogramar	Chat VialAI: "¿Sigo o me desvío?" con respuesta accionable

Cuadro 13: Perfiles de usuario de VialAI y sus necesidades operativas específicas.

11.3 Accionables habilitados por VialAI

VialAI no entrega "un mejor GPS": entrega *decisiones operativas accionables* que los sistemas puntuales no pueden generar. Los cuatro accionables principales son:

1. **Quoting de ventanas de entrega.** En lugar de prometer "tu paquete llega entre las 14:00 y las 18:00" (ventana de 4 horas genérica), el dispatcher puede prometer "entre las 15:30 y las

16:10" basado en $P_{50}\pm$ media banda. Esto reduce ventanas prometidas en ∼60 % y mejora la experiencia del cliente.

2. **Priorización dinámica de rutas.** Al inicio del turno, el sistema rankea las rutas del día por IC: rutas verdes se asignan a conductores nuevos (menor riesgo operativo), rutas rojas a conductores experimentados. Esto reduce fallas de entrega por variabilidad.

3. **Alertas tempranas por IC alto.** Cuando una ruta crítica escala a IC > 0.60, VialAI notifica proactivamente al dispatcher sugiriendo: (a) dividir la carga, (b) reasignar a otro vehículo, o (c) comunicar al cliente el posible retraso antes de que ocurra.

4. **Reporte post-mortem de entregas.** Al final del día, VialAI genera un reporte comparando IC predicho vs. tiempo real observado, identificando patrones (ej. "las rutas hacia Santa Fe los jueves por la tarde consistentemente superan el P90"), lo que alimenta la recalibración continua del modelo.

11.4 Propuesta de valor y modelo de entrega

VialAI se entrega bajo un modelo **SaaS freemium** inspirado en el patrón de herramientas B2B exitosas en LATAM:

- **Tier Gratuito (gancho):** Acceso al dashboard web con hasta 50 consultas/día. Permite al dispatcher probar el sistema sin fricciones de procurement. Este tier es el equivalente al "reporte básico" del profesor Barranco en su sistema Yelp.
- **Tier Starter (PyME, $1,500 MXN/mes):** Consultas ilimitadas, API REST para integración con TMS existente, 3 usuarios.
- **Tier Pro (operador mediano, $4,500 MXN/mes):** Todo lo anterior + alertas por webhook, reporte post-mortem diario, SLA 99.5 % de uptime, soporte prioritario.
- **Tier Enterprise (negociado):** Modelo calibrado con datos propios del cliente, integración custom, on-premise opcional.

Diferenciador competitivo

Los competidores actuales (Google Maps, Waze, TomTom, OSRM) entregan *estimaciones puntuales deterministas*. Ninguno entrega bandas probabilísticas P10/P90 con explicación contextual en lenguaje natural específico para el contexto mexicano. VialAI es, hasta donde sabemos, el único sistema que combina: **(1)** predicción estocástica, **(2)** perturbaciones contextuales específicas de la ZMVM (cierres Metro, manifestaciones) y **(3)** interfaz conversacional en español. Esta triple combinación crea un foso defendible frente a soluciones genéricas.

12 Conclusiones y Trabajo Futuro

12.1 Conclusiones

1. **La modelación estocástica supera al enfoque determinístico.** Las bandas de incertidumbre P_{10}/P_{90} proveen información accionable que un tiempo puntual no puede ofrecer. La validación empírica confirma un MAPE del 11.9 % (bajo el objetivo del 15 % y 7.1 puntos porcentuales mejor que TomTom ETA) y una cobertura de banda del 93.3 %, ambos indicadores de un modelo bien calibrado y conservador.

2. **Las cadenas de Markov de 3 estados son un modelo adecuado para la dinámica del tráfico urbano.** La propiedad de Markov es una aproximación razonable: el estado vial en $t+1$ está correlacionado principalmente con el estado en t. Los elementos diagonales dominantes en $\hat{P}$ confirman la inercia del tráfico observada en la ZMVM.

3. **Monte Carlo con $N = 10{,}000$ es suficiente.** El error estándar del estimador del P_{50} es ± 0.5 min, con tiempo de cómputo menor a 500 ms en CPU sin GPU. La vectorización por transformada inversa fue esencial para alcanzar esta latencia.

4. **Las perturbaciones contextuales son la pieza que falta en los sistemas comerciales.** Eventos como cierres de Metro o manifestaciones no son capturados por TomTom ni Google Maps, pero tienen impactos de decenas de minutos en el tiempo de viaje. Inyectar este conocimiento al motor vía Ω es un diferenciador importante.

5. **El patrón agente LLM + motor determinista es poderoso.** Separar el motor estocástico (determinista una vez fijadas las semillas, testeable, reproducible) del agente conversacional (interpretativo, flexible, en lenguaje natural) permite iterar sobre cada capa de forma independiente.

6. **El nicho de logística B2B de última milla es la mejor aplicación.** En este sector, la predictibilidad vale más que la velocidad promedio, y el costo de la incertidumbre es cuantificable en pesos por día.

12.2 Retos encontrados durante el desarrollo

Más allá de los retos técnicos específicos ya documentados en las secciones anteriores (indisponibilidad de API, probabilidades cero, vectorización, bug del .env), merece la pena destacar tres desafíos metodológicos transversales:

1. **Validación sin ground truth perfecto.** TomTom Routing no es un oráculo: su propio ETA tiene error. Se decidió usarlo como proxy de validación aceptando que el MAPE reportado es una *cota superior* del error verdadero.

2. **Balance entre complejidad del modelo y datos disponibles.** Hubo tentación de agregar estados, cadenas jerárquicas o modelos de Markov no homogéneos. Se decidió mantener la simplicidad de 3 estados estacionarios porque los datos del C5 CDMX no son suficientes para identificar más parámetros de forma confiable.

3. **Comunicación de incertidumbre a usuarios no técnicos.** Los percentiles son difíciles de interpretar para un dispatcher sin formación estadística. El Índice de Confiabilidad con semáforo verde/amarillo/rojo fue la solución: un número adimensional (IC) y un código visual que captura todo el concepto en 2 segundos.

12.3 Trabajo futuro

Los siguientes pasos priorizados por impacto son:

1. **Profundización de la detección en tiempo real.** La versión actual del módulo de eventos (§8.4) consulta la API CKAN del C5 CDMX en modo pull con caché de 5 minutos. El siguiente paso es migrar a streaming vía webhooks o Server-Sent Events para reducir la latencia de detección de $\sim$5 minutos a $<$30 segundos. También se plantea incorporar feeds de redes sociales con geolocalización y alertas de Protección Civil CDMX.

2. **Matrices diferenciadas por corredor y franja horaria.** Una matriz $\hat{P}$ global subestima la heterogeneidad. Entrenar matrices específicas por corredor (Periférico, Insurgentes, Reforma) y por hora (pico AM, valle, pico PM) debe reducir el MAPE por debajo del 10 %.

3. **Calibración bayesiana de la matriz.** Reemplazar el MLE con suavizado de Laplace por un estimador bayesiano con priori informativo sobre los p_{ij} debe mejorar el comportamiento en corredores con pocos datos.

4. **Agente multi-turno con memoria.** Actualmente VialAI es stateless entre sesiones. Agregar memoria permitiría planeación de viajes recurrentes ("mi ruta de cada martes") y aprendizaje personalizado.

5. **API REST con FastAPI.** Para habilitar la integración con TMS de clientes B2B, necesitamos exponer el motor como servicio REST con autenticación, rate-limiting y documentación OpenAPI.

6. **Expansión a otras Zonas Metropolitanas.** Guadalajara, Monterrey y Puebla tienen dinámicas de tráfico análogas; la arquitectura del sistema es replicable cambiando solo el corpus de entrenamiento de la matriz $\hat{P}$.

Referencias

[1] Data México (2024). *Valle de México: Perfil económico, demográfico y social.* Secretaría de Economía, Gobierno de México. https://www.economia.gob.mx/datamexico/es/profile/geo/valle-de-mexico

[2] COESPO (2019). *Características demográficas de la Zona Metropolitana del Valle de México.* Consejo Estatal de Población, Estado de México.

[3] TomTom International BV (2025). *TomTom Traffic Index 2024: Mexico City Report.* https://www.tomtom.com/traffic-index/mexico-city-traffic/

[4] Milenio (2025). "CDMX entre las ciudades con más tráfico del mundo en 2024: TomTom." https://www.milenio.com/comunidad/cdmx-entre-las-ciudades-con-mas-trafico-del-mundo-en-2024-tomtom

[5] La Silla Rota (2025). "CDMX: Conductores pierden 152 horas al año atorados en embotellamientos."

[6] Eje Central (2025). "TomTom Traffic Index 2025: velocidad de 5 km/h y 152 horas perdidas al año en CDMX."

[7] TomTom International BV (2025). *TomTom Traffic Index 2024.* https://www.tomtom.com/traffic-index/mexico-city-traffic/

[8] Mordor Intelligence (2025). *Mexico Last Mile Delivery Market — Size, Share and Industry Analysis 2025–2030.* https://www.mordorintelligence.com/industry-reports/mexico-last-mile-delivery-market

[9] T21 (2026). "Última milla, el eslabón más crítico de la logística en México hacia 2030."

[10] The Logistics World (2024). "Logística de última milla: innovaciones para entregas rápidas y eficientes en 2025."

[11] Mail Boxes Etc. (2024). "Logística digital de última milla, componente estratégico del e-commerce en México."

[12] Norris, J. R. (1997). *Markov Chains.* Cambridge University Press.

[13] Rubinstein, R. Y., & Kroese, D. P. (2016). *Simulation and the Monte Carlo Method* (3rd ed.). Wiley.

[14] Van Lint, J. W. C., & Hoogendoorn, S. P. (2010). "A Robust and Efficient Method for Fusing Heterogeneous Data from Traffic Sensors." *Computer-Aided Civil and Infrastructure Engineering*, 25(8), 596–612.

[15] SEMOVI (2023). *Aforos vehiculares en la Zona Metropolitana del Valle de México*. Secretaría de Movilidad, Gobierno de la Ciudad de México.

[16] Anthropic (2024). *Tool use with Claude*. Anthropic Documentation. https://docs.anthropic.com/en/docs/build-with-claude/tool-use

[17] Harris, C. R. et al. (2020). "Array programming with NumPy." *Nature*, 585, 357–362.

[18] Streamlit (2024). *Streamlit Documentation*. https://docs.streamlit.io

A Parámetros de configuración del motor

Parámetro	Valor	Descripción
`n_simulaciones`	10,000	Número de trayectorias Monte Carlo
`paso_minutos`	1.0	Resolución temporal Δt (min)
`max_pasos`	480	Horizonte máximo (8 horas)
`suavizado`	10^{-6}	Constante de Laplace ε
`FACTOR_TORTUOSIDAD`	1.4	Factor κ Haversine $\rightarrow$ carretera
`UMBRAL_FLUIDO`	0.75	Ratio mínimo para estado Fluido
`UMBRAL_LENTO`	0.45	Ratio mínimo para estado Lento
`UMBRAL_IC_VERDE`	0.30	IC máximo para ruta confiable
`UMBRAL_IC_AMARILLO`	0.60	IC máximo para ruta volátil

Cuadro 14: Parámetros configurables del motor estocástico.

B Glosario

Cadena de Markov Proceso estocástico donde el estado futuro depende únicamente del estado presente.

Distribución estacionaria (π^*) Vector de probabilidades al que converge la cadena en el largo plazo.

Índice de Confiabilidad (IC) Métrica $(P_{90} - P_{10})/P_{50}$ que resume la volatilidad relativa de una ruta.

Monte Carlo Método de estimación basado en muestreo aleatorio masivo.

Normal truncada Distribución normal restringida a un intervalo $[v^{\min}, v^{\max}]$.

$P_{10}/P_{50}/P_{90}$ Percentiles de la distribución simulada de tiempos de viaje.

Ratio de congestión (r) Cociente $v_{\text{actual}}/v_{\text{libre}}$; mide el grado de congestión.

MAPE Mean Absolute Percentage Error; error porcentual absoluto medio.

ZMVM Zona Metropolitana del Valle de México.

Structured Outputs Patrón donde un LLM devuelve JSON validado por un esquema.

Tool use loop Ciclo agente-herramienta en el que el LLM invoca funciones externas.

Perturbación contextual Transformación de la matriz de transición que inyecta conocimiento sobre eventos externos.